

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Lectura #1 - Semana 2 – Capítulo 1
Introducción al Lenguaje Ensamblador

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 05/08/2024

Bibliografía:

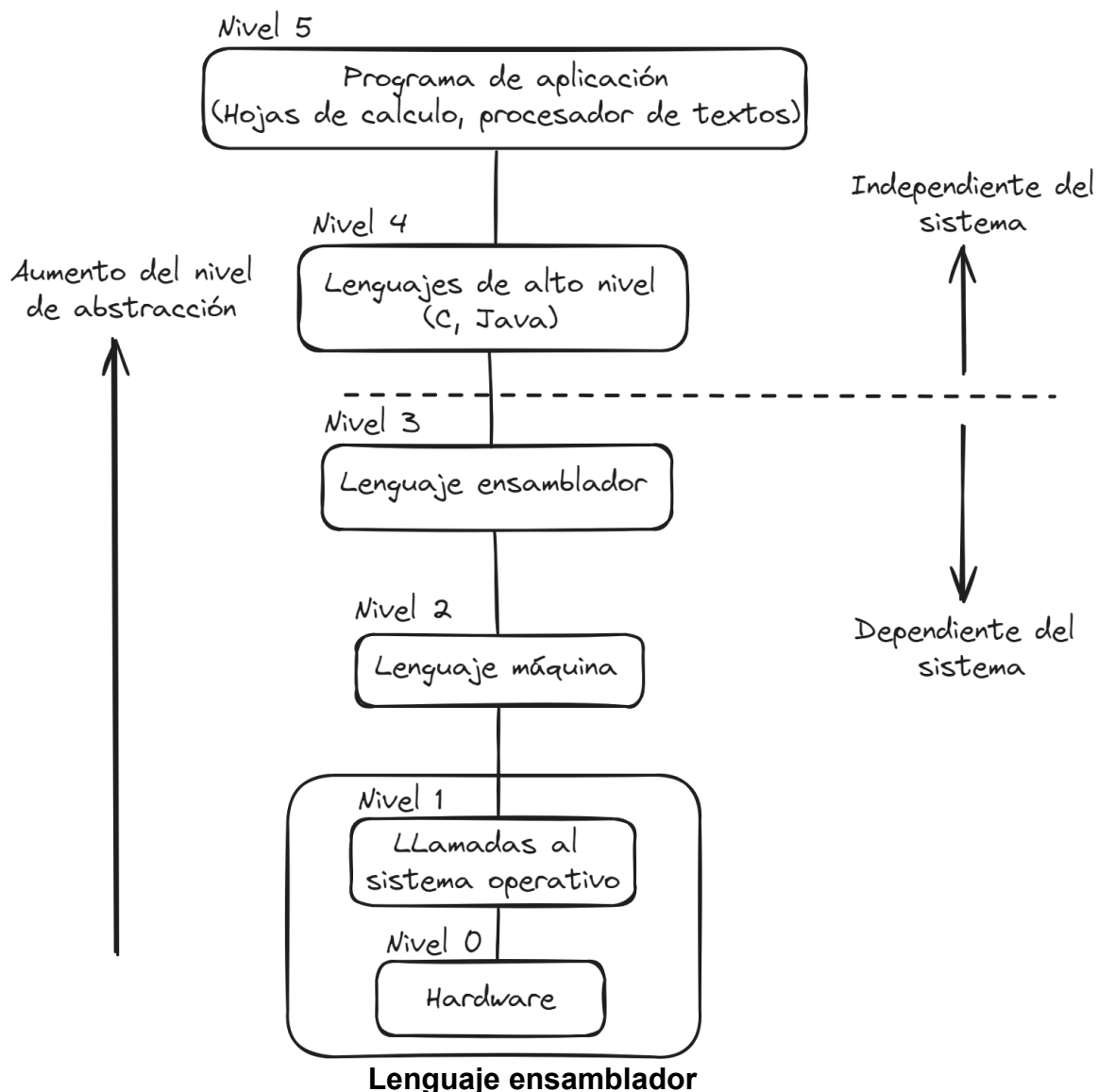
[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*.
Springer Science & Business Media, 2005, pp. 3–8.

Abstract—*User interaction with computer systems varies with the level of software abstraction. At higher levels, users work with applications without technical details, while lower levels, like high-level languages, require more in-depth software knowledge. High-level languages are system-independent and portable, whereas assembly language is processor-specific and needs more code. Although assembly language is low-level, it abstracts some system details and relies on the operating system for complexity management, with instructions executed directly by hardware. While high-level languages offer faster development, easier maintenance, and better portability, assembly language provides compact, efficient code and precise hardware control, making it essential for tasks such as device driver development and embedded systems.*

Introducción al lenguaje ensamblador

La percepción del sistema informático por parte del usuario varía según el nivel de abstracción del software. En el nivel más alto, los usuarios interactúan con el sistema sin necesidad de conocer detalles técnicos. A niveles más bajos, como con lenguajes de programación de alto nivel como C o Java, los programadores necesitan entender aspectos del desarrollo de software, aunque no necesariamente el funcionamiento interno del sistema.

Los lenguajes de alto nivel permiten que los programas sean independientes del sistema, pudiendo ejecutarse en diferentes procesadores con solo recompilarlos. En cambio, la programación en ensamblador, de bajo nivel, es específica del procesador y requiere más código para realizar las mismas tareas. Aunque el lenguaje ensamblador es de bajo nivel, el sistema operativo oculta muchos detalles internos complejos, mientras que las instrucciones en lenguaje de máquina se ejecutan directamente en el hardware del sistema.



El lenguaje ensamblador está diseñado según el conjunto de instrucciones y la arquitectura del procesador. Para convertir el código en ensamblador (assembly) a lenguaje de máquina, se utiliza un programa llamado ensamblador (assembler).

El ensamblador NASM (Netwide Assembler) es una opción destacada debido a que es gratuito y compatible con varios formatos, incluyendo los de Windows y Linux.

Comparación de instrucciones:

Instrucción Ensamblador	Descripción	Código Máquina (Hexadecimal)
nop	No operación	90
inc result	Incrementar result	FF060A00
mov class_size,4	Copiar 4 a class_size	C7060C002D00
and mask, 128	AND lógico con 128	8,03E+84
add marks, 10	Suma 10 a marks	83060F000A

Ventajas de los lenguajes de alto nivel

1. **Desarrollo rápido:** Ofrecer estructuras que simplifican la codificación y depuración lo que aumenta la velocidad de desarrollo, y sus programas son más pequeños que los escritos en ensamblador.
2. **Mantenimiento más fácil:** Son más fáciles de entender y mantener, lo que facilita la corrección de errores y la actualización del software, en comparación con los programas extensos y complejos en ensamblador.
3. **Portabilidad:** Son menos dependientes del procesador, lo que permite su uso en diferentes sistemas con pocas o ninguna modificación, a diferencia de los programas en ensamblador, que son específicos del procesador.

Ventajas del lenguaje ensamblador

1. **Eficiencia:** Los programas en ensamblador suelen ser más compactos, lo que significa que ahorran memoria en relación con el tamaño del código ejecutable. Además, pueden ejecutarse más rápido al estar optimizados específicamente para el hardware en el que se ejecutan.
2. **Acceso directo al hardware:** El lenguaje ensamblador permite un control preciso del hardware, algo esencial para tareas como desarrollar controladores de dispositivos o sistemas embebidos. Estas tareas a menudo no son posibles con lenguajes de alto nivel, que abstraen detalles del hardware.

